

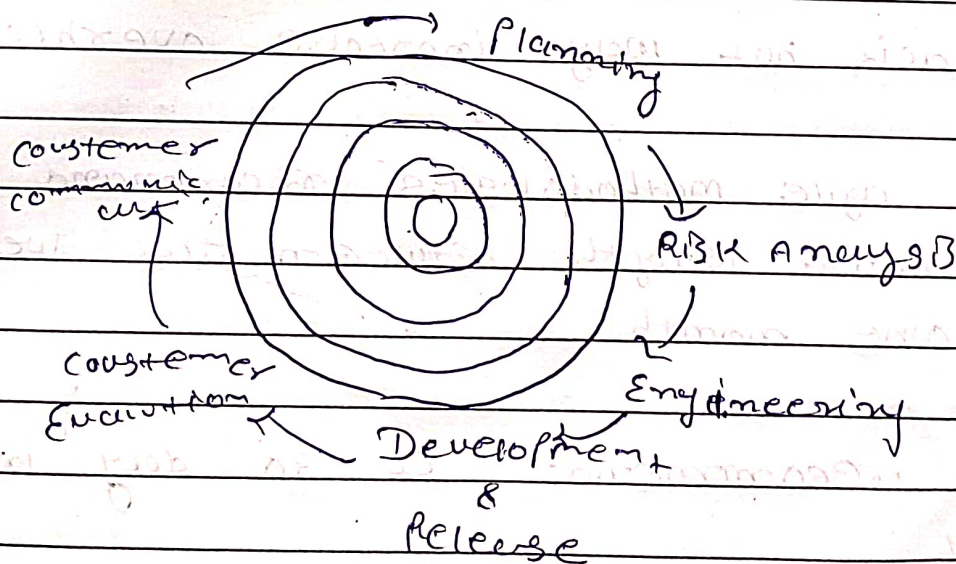
→ Reflex Solution -

2) Answer the following.

Q. XP Phases -

- Exploring Phase.
- Planning Phase
- Iteration and release Phase.
- Productionizing Phase
- maintenance Phase
- Death Phase

b) Spiral Development Model -



Working -

→ The radius of the spiral model the represent cost of project and the angular degree represent the progress

Date _____
Page _____

made in current phase, each phase begins with goal for the design and ends when the developer or client review the progress.

Every phase can be broken into four quadrants: identifying and understanding requirements, performing risk analysis, building the prototype, and evaluation of the software performance.

C) Key consideration on deciding increment length for scrum projects.

- shorter increments provide more frequent feedback but may increase overhead.
- most agile methodologies recommend an increment length between two weeks and one month.
- scrum recommends a 30 day increment length.

cons -

b) complexity of the solution

- High complicated solution may require more time to coordinate deployments
- Deployment automation is critical.

c) maturity of the team -

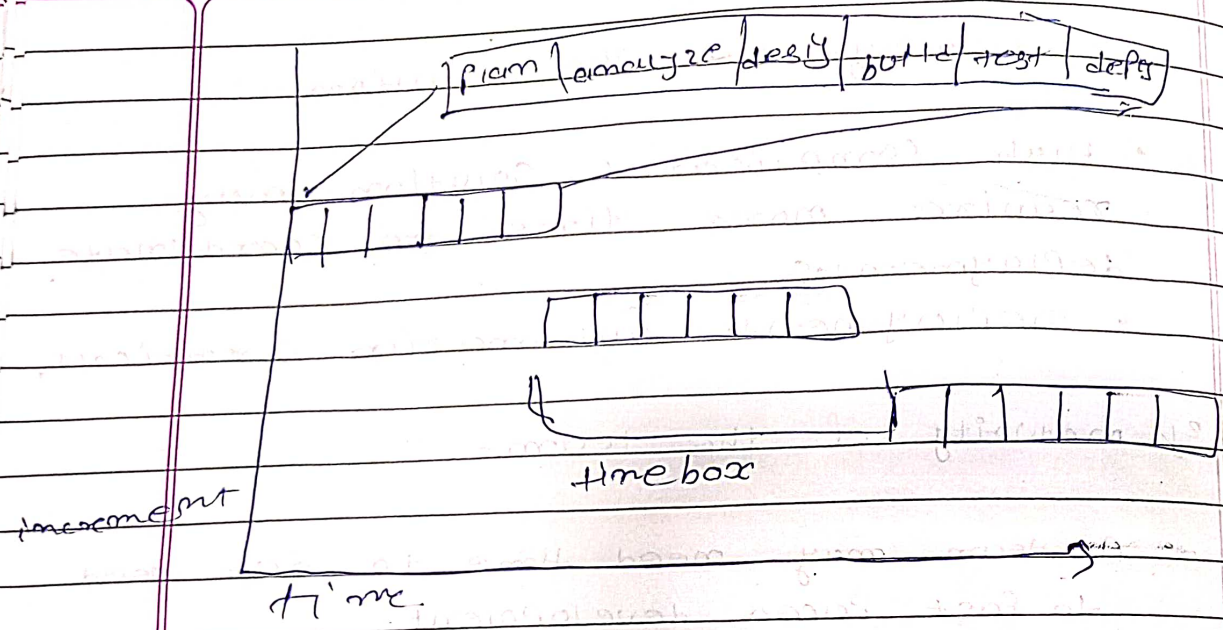
- A team may need time to get used to fast paced development.
- shorter increments may increase pressure which can lead to a "throw it over the wall" mentality

3) over all Project timeline -

- > If the development time is less than a month it's usually better to go with 2 week increments.

d) Flexibility -

Agile teams are flexible and adaptable to changing requirements and prioritize responding to change over sticking to a rigid plan.



- e)
- 1) Project running over - budget
 - 2) Project running over - time.
- Software was very inefficient
 software was of low quality.
 software often did not meet requirements
 Project were unmanageable and ^{difficult to maintain} code tests

3) Adopting Agile methodologies -> Agile focuses on iterative development, adapting to changing requirements effectively. Enhanced skill Development, continuous learning and development can equip professionals with up-to-date skills and knowledge

- 1) Over-specification
- 2) contradiction
- 3) ambiguity
- 4) forward Reference
- 5) wishful Thinking
- 6) unstructured specification
- 7) Noise
- 8) Silence

9) Key objective SA/SD

SA/- capture the detailed structure of the system as the user view it.

SD/- Arrive at a form that is suitable for implementation in some programming language.

10) online-

1) significant of Data Dictionary

Provides a standard terminology for all data in a project

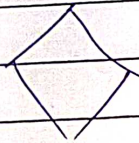
A consistent vocabulary for all data in a project
A consistent vocabulary / for data / is very important.

- Different team member tend to use very important different terms to refer to the same data it cause unnecessary confusion.

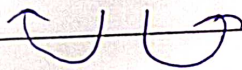
1. U module



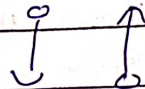
if condition



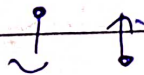
3) loop



By Data Flow



5) Control Flow



Input from end-user
customers, team and
other stakeholders

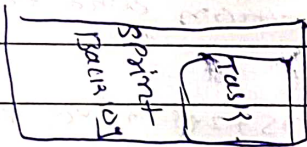


Product owner

1	
2	
3	
4	
5	
6	
7	
8	
9	

Team selects
How much to
commit to do
by sprints end
sprints planning
meeting

0	0	0	0	0
n	n	n	n	n
Team				



Product
Backlog
Refinement

Review
meeting

Sprint
4 weeks

No changes
in Duration or scope

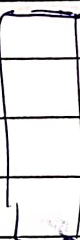


Daily Scrum
meeting
with participants

0	0	0	0	0
n	n	n	n	n

0	0	0	0	0
n	n	n	n	n
Review				

Productivity
shippable
product
Increment



Product

Backlog

Date

Page

b) give Phases of cocomo model -

Planning & requirements



system Design



detailed Design



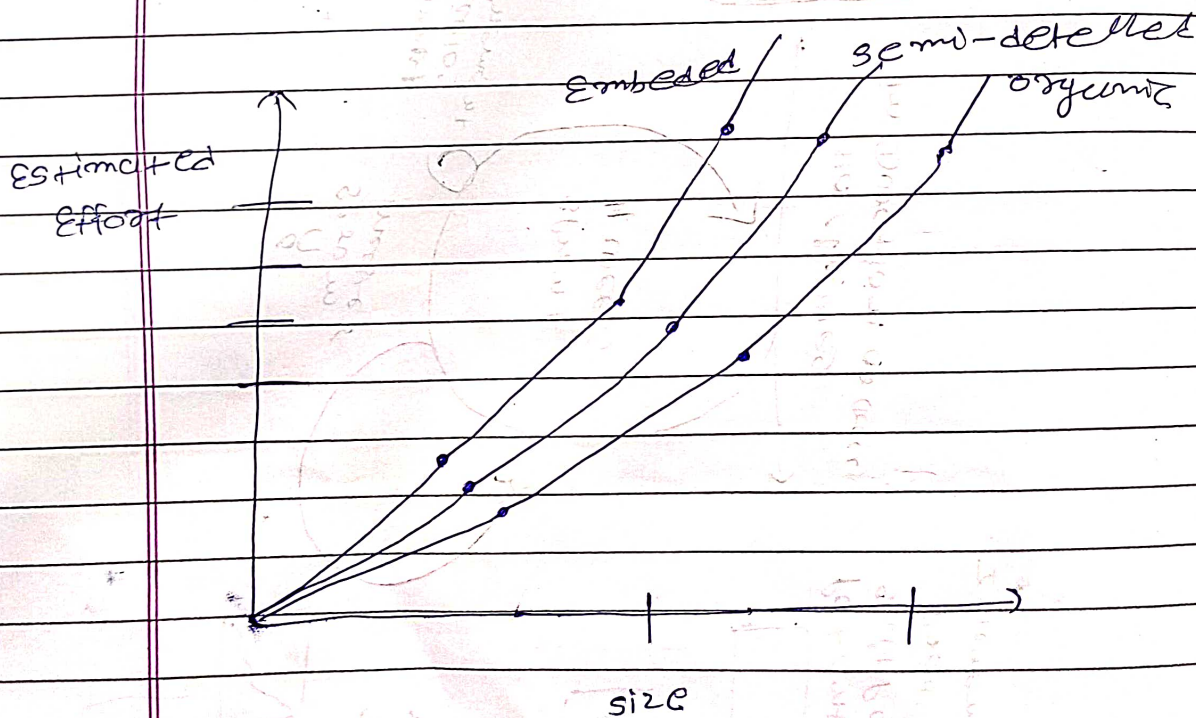
module code & test



Integration and test



cost constructive model

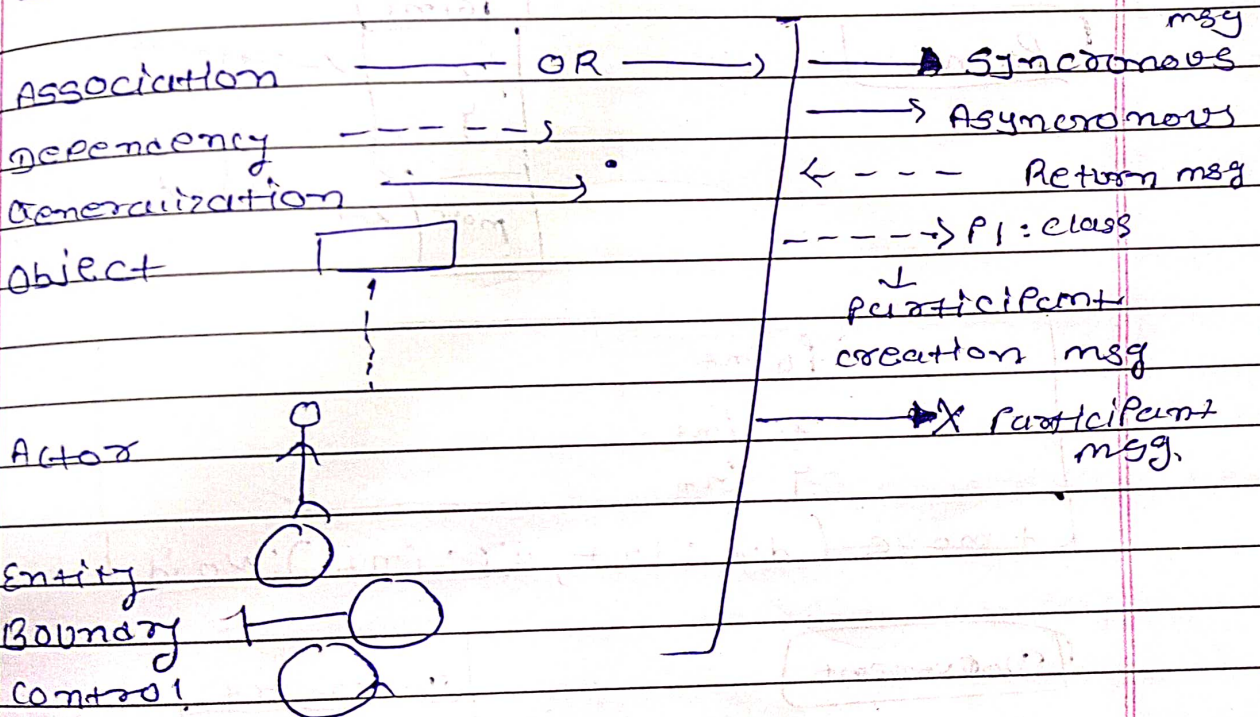


Answer the following

a) User-Interface Principle -

- user familiarity
- consistency
- minimum surprise
- recoverability
- user guidance

b) UML sequence diagram notation



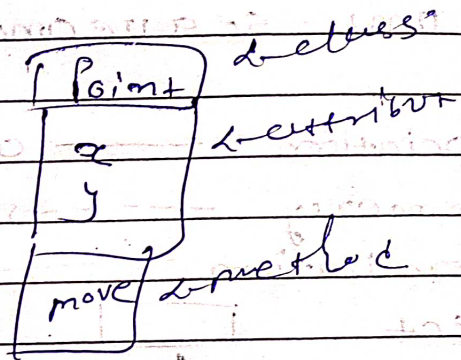
+ → public
- → private

C/ Class Diagram abstraction-

class name	→ The top component shows the class name
field 1	The middle component contains the declaration of the field or attributes of the class
field n	
method 1	
method n	The bottom component contains the declaration of the method of the class

Example

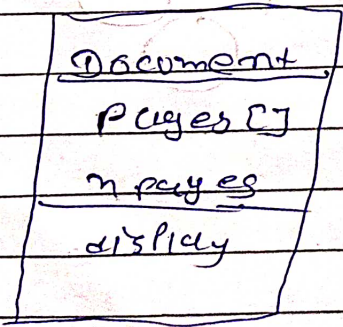
Point



```

class Point {
    x: int
    y: int
    +move(dx: int, dy: int): void
}
  
```

Document



```

class Document {
    -pages: array of str
    -n pages: int
    +display(x: int, y: int) void
}
  
```


d) interaction occurrences and combine fragments equivalent to when to typical programming construct?

→ interaction occurrence

- Reusable interaction
- Using ref elements in the sequence diagram
- To accommodate long sequence diagram by invoking a SD from another SD.

combined fragments - for selection and loop in SD

→ Alt (equivalent of if)

→ opt (equivalent of if-else)

→ loop (equivalent of loop)

e) different various type of testing

→ Unit testing

→ Integration testing

→ System testing

→ Acceptance testing

f) condition coverage & branch coverage

Condition coverage technique -

→ It is used to cover all conditions.

→ condition coverage is also known as predicate coverage in which each one of the Boolean expression have been evaluated to both TRUE and FALSE.

example -

read a, b, c;

if (a == 0 || b == 0)

{

Print 7;

}

else

{

if (c == 0 && d == 0)

{

Print 8;

}

We have check and cover all the condition one by one putting the true or false value.

Branch Box testing.

Branch coverage testing is a software testing technique that aims to ensure that every possible decision point in a code is tested to help identify potential logic flaws in test coverage.

Branch coverage testing aims to validate all possible routes or outcomes resulting from conditional statement.

STEP-1 Identifying Decision Point

1. If (score $\geq$ 90)
2. else if (score $\geq$ 80)
3. else if (score $\geq$ 70)
4. else if (score $\geq$ 60)
5. else

STEP 2. creating Test cases -

Test case 1 - score of 95 (Grade 'A')
like 5 test case.

STEP 3 Test execution

Enter the student score '95'
The student grade is 'A'

STEP - 4 Branch Coverage Analysis

In test case 1, the branch for 'A' grade is taken like all.

STEP-5 Review & Analysis

All the branches are covered or not.

STEP 6 - completion and Reporting

Branch coverage ensure that all branches are exercised while condition coverage ensure that both true and false outcomes of individual condition are tested.

g. Boundary Value Testing -

The black box testing techniques are helpful for detecting any error or threats that happened at the boundary value of valid or invalid positions rather than focusing on the center of the input data.

In black box testing we do not look at the internal structure, we

Test case Number #1

Let us assume a test case that the speed of a car from 40 to 80 to get the best fuel efficiency.

Boundary Value Testcase

Invalid ^{case} Test	Valid ^{case} Test	Invalid ^{case} Test
(min - 1)	(min, +min, max)	(max + 1)
39	40, 41, 79, 80	81

Equivalence Partition testing

- It is a black box testing method
- All possible test cases that we cannot try them all
- we have to select a relatively small number of test cases to actually run.
- we do equivalence partitioning help in this case.

- Input/Outputs / internal value of the software are divided into groups that are expected to exhibit similar behavior.

example

People (Woman / man)

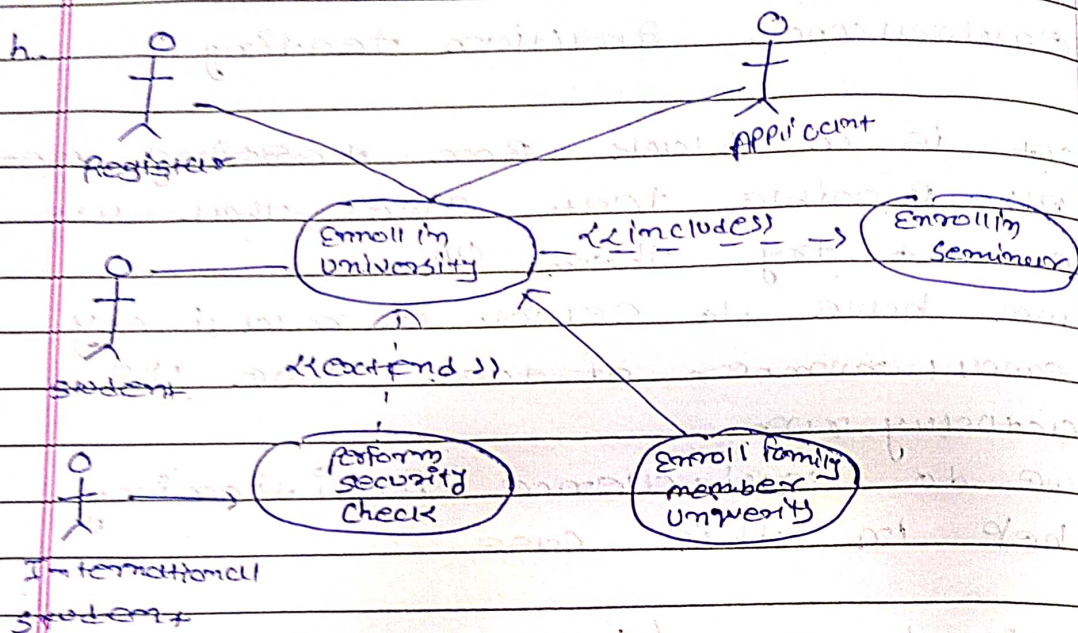
Students (bachelor / master)

Tickets (children / youth / adults / old)

Vehicle (gasoline / diesel / petrol)

- we check only one item per group.

- EP focuses on equivalence classes while BVA emphasize boundary condition test.



→ If student is enroll in university is the initial involve in enroll in seminars.

→ If the international student only then extend the perform security check and enroll in university.

«include» The include relationship adds additional functionality not specified in the base use case. The «include» relation is used to include common behavior from an include use case or a base use case in order to support of common behaviour.

Extend >>

In UML modelling you can use a extend relation to specify that one use case (extension) the behavior another use case (base).

→ This type of relationship reveals details about a system or application that are typically in a use case.

i. The expert judgment in Project management refer to relying on knowledge specificity to provide informed opinions on key project factors like timelines, cost, risk

When managing project you often need to make critical decision without having complete information or the required knowledge. That's where expert judgment comes in.

Cocoma estimation model,
(Constructive cost model)

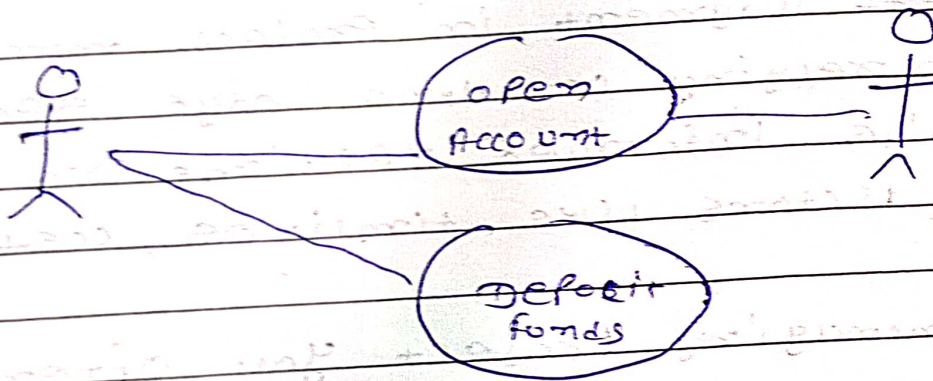
- Organic mode
- Semi-detached mode
- Embedded mode

i. Association ————— OR —————
Permanent, structural, "has a"

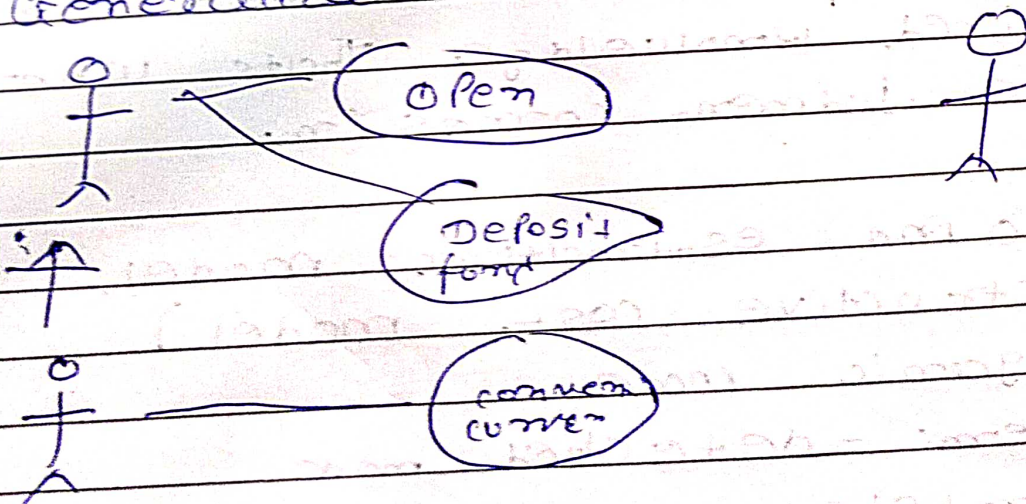
Dependency - - - - -> (include <
Temporary "uses a" <<extend>>)

Generalization —————>
Inheritance "is a"

Association-



Generalized



5.6 cyclomatic complexity.

